

Resumen Teórico – Clases y Herencia en JavaScript

1. Clases en JavaScript

Las clases en JavaScript son una forma de crear objetos utilizando un modelo basado en prototipos, pero con una sintaxis más clara y parecida a otros lenguajes orientados a objetos. Una clase define las propiedades (variables) y métodos (funciones) que tendrán los objetos creados a partir de ella.

2. Constructor

El constructor es un método especial que se ejecuta automáticamente cuando se crea una nueva instancia de una clase. Se utiliza para inicializar los valores iniciales del objeto. Dentro del constructor se usa la palabra clave **this** para referirse al objeto actual.

3. Métodos de una clase

Los métodos son funciones definidas dentro de una clase. Permiten definir el comportamiento de los objetos. Se accede a ellos a través de las instancias creadas con **new**.

4. Instanciación de objetos

Una instancia es un objeto creado a partir de una clase. Se crea utilizando la palabra clave **new**, seguida del nombre de la clase y los parámetros requeridos por el constructor.

5. Herencia (extends)

La herencia permite crear una clase hija a partir de una clase padre. La clase hija hereda las propiedades y métodos de la clase padre, lo que evita duplicar código y mejora la reutilización.

6. super()

La palabra clave **super()** se utiliza dentro del constructor de una clase hija para llamar al constructor de la clase padre. Es obligatoria antes de usar **this** en una clase que hereda de otra.

7. Sobrescritura de métodos (override)

La sobrescritura de métodos ocurre cuando una clase hija define un método con el mismo nombre que uno de la clase padre. Esto permite modificar o extender el comportamiento original del método heredado.

8. Relación entre Persona y Estudiante

En el ejemplo, **Persona** es la clase base que define nombre, edad y un método saludar. **Estudiante** hereda de Persona, agrega la propiedad curso y redefine el método saludar para incluir información adicional.

Este tipo de estructura es muy común en JavaScript moderno y es fundamental para entender la Programación Orientada a Objetos (POO).